Scam Variant Observatory

System documentation: architecture, system flow, machine learning pipeline and operations

Project: ScamFit / TensorForge 2.0 (Lumora)
Served model: svo-e2a46f504b19 (v2, e5_tfidf)
Repository branch documented: feature/frontend
Document date: 17 October 2026

Scope: private, unreviewed demonstration. Not validated fraud protection. Low risk does not mean safe.

Contents

1. Introduction 4

1.1 What the system does . . . 4

1.2 Main components . . . 4

1.3 Key facts at a glance . . . 4

2. Repository layout 6

3. System architecture 7

3.1 Deployment view . . . 7

3.2 Architectural principles . . . 7

3.3 Process and thread model . . . 8

4. System flow 9

4.1 Startup sequence . . . 9

4.2 Prediction request lifecycle . . . 9

4.3 Scoring and decision logic . . . 10

4.4 Monitoring flow: novelty, clusters and drift . . . 11

4.5 Analyst review flow . . . 11

5. API reference 13

5.1 Endpoints . . . 13

5.2 Request and response examples . . . 13

5.3 Validation rules . . . 13

5.4 Error envelope and codes . . . 14

6. Data storage 15

6.1 SQLite schema . . . 15

6.2 Data that stays on the analyst device . . . 15

7. Security and privacy 16

7.1 Request boundary . . . 16

7.2 Modes . . . 16

7.3 Container hardening . . . 16

7.4 Artifact integrity . . . 16

8. Analyst console (frontend) 17

8.1 Pages . . . 17

8.2 Design decisions . . . 17

9. Machine learning pipeline 19

9.1 Data sources . . . 19

9.2 Data preparation and splits . . . 19

9.3 Training (svo.train_v2) . . . 20

9.4 Evaluation (svo.evaluate_v2, once only) . . . 20

9.5 Results . . . 21
9.6 Why v2 replaced v1 . . . 21
10. Deployment and operations **22**
10.1 Running the system . . . 22
10.2 Configuration . . . 22
10.3 Backup, restore and rollback . . . 22
10.4 Scaling constraints . . . 23
11. Testing and verification **24**
11.1 Verification status (v2, 17 October 2026) . . . 24
12. Limitations and next steps **25**
12.1 Known limitations . . . 25
12.2 Recommended next steps . . . 25
12.3 Glossary . . . 25

1. Introduction

The **Scam Variant Observatory** (SVO) scores text messages sent to Sri Lankan users for scam risk, watches the stream of scored messages for wording the model has not seen before, groups recurring unfamiliar messages into candidate *variants*, and gives analysts an audited place to confirm or reject them. The design follows the problem statement's **detect** → **monitor** → **respond** loop, with deployment, monitoring and drift response weighted above raw model accuracy.

1.1 What the system does

Stage	What happens	Where
Detect	Each message receives a calibrated scam score and one of three decisions: **high_risk**, **review** or **low_risk**.	Model bundle, `svo.model.Runtime`
Monitor	Each message also receives a novelty score (distance to the nearest known example). Unfamiliar messages are flagged; rolling windows raise a drift alert when unfamiliar messages stay elevated.	`svo.monitor.Monitor`, SQLite
Group	Messages routed to review are clustered by similarity. A cluster with enough distinct, novel texts becomes a **candidate variant**.	Cluster table, analyst queue
Respond	Analysts review clusters (pending / confirmed / rejected). Every decision is stored in an audit trail. Confirmation never retrains the model automatically.	Review API, React console

1.2 Main components

Component	Technology	Location
Training pipeline	Python 3.12, scikit-learn, sentence-transformers (multilingual-E5-small), NumPy/SciPy	`training/`
Frozen model bundle	joblib model, decision config, model card, E5 encoder, SHA-256 manifest	`training/runs/v2/candidates/e5_tfidf/bundle/`
API service	FastAPI, Uvicorn, Pydantic (strict), SQLite in WAL mode	`app/server/`
Analyst console	React 19, React Router 7, Tailwind CSS 4, Vite 6, TypeScript	`app/client/`
Deployment	Docker Compose: `api` service plus optional `client` (nginx) behind the `frontend` profile	`compose.yaml`
Planning and rules	Problem statement, finale rules, MLOps plan, API plan	`Resources/`

1.3 Key facts at a glance

Item	Value
Served model version	`svo-e2a46f504b19` (representation `e5_tfidf`, C = 256)
Languages in data	English, Sinhala, Tamil, Singlish, Tanglish
Held-out macro-F1	0.932 (95% group bootstrap CI 0.88 to 0.99)
High-risk precision / recall (held-out)	83.2% / 98.8% (precision target of 90% not met)
Legitimate false-positive rate	9.8% overall; 1.4% on real SMS
Review workload	40 per 100 messages
Drift detection on held-out shift	Not detected (no alert, no candidate cluster)
API port / console port	127.0.0.1:8000 / 127.0.0.1:8080
Memory	About 1.2 GB resident; Compose limit 2 GiB

> **Version note.** The root `README.md` still describes the earlier v1 model (character TF-IDF, `svo-38879254a924`, bundle at `training/runs/final/bundle/`). The Dockerfile, settings and server README now serve v2 (`svo-e2a46f504b19`). This document describes the v2 system that the code actually runs. v1 is covered as history in section 9.

2. Repository layout

```
ScamFit-TnsorForge-Lumora/
├── README.md                 project entry point (describes v1; see version note)
├── compose.yaml              api service + optional client service (profile "frontend")
├── .dockerignore             allowlist build context: server code, runtime svo modules, v2 bundle
├── Resources/                problem statement, finale rules, MLOps plan, API plan
├── app/
│   ├── server/               FastAPI service ("svo_api")
│   │   ├── Dockerfile        python:3.12-slim (pinned digest), non-root, read-only model
│   │   ├── requirements*.in / *.lock
│   │   ├── src/svo_api/
│   │   │   ├── main.py       app factory, lifespan, error handlers, OpenAPI security
│   │   │   ├── auth.py       ASGI "Boundary": auth, body limits, request IDs, logs
│   │   │   ├── settings.py   SVO_* environment configuration and validation
│   │   │   ├── schemas.py    strict Pydantic request/response models
│   │   │   ├── errors.py     APIError and the JSON error envelope
│   │   │   ├── routes/       health, predictions (+ model), monitoring, clusters
│   │   │   ├── services/     engine.py (queue + thread), monitoring.py (DurableMonitor)
│   │   │   └── storage/      migrations.py (schema version 1)
│   │   ├── scripts/          smoke, benchmark, backup, verify_runtime, offline_smoke
│   │   └── tests/            contract, parity, persistence, container smoke
│   └── client/               React analyst console
│       ├── Dockerfile, nginx.conf
│       └── src/  api/  lib/  components/  pages/
└── training/
    ├── src/svo/              data, preprocessing, model, monitor, artifacts, v2, train_v2, evaluate_v2
    ├── scripts/              prepare_data, add_public_controls, download_encoder, predict, verify_bundle
    ├── configs/              experiment.json (v1), experiment_v2.json (v2)
    ├── data/                 private seeds, external UCI SMS, processed JSONL + audits
    ├── runs/                 baseline, final (v1), v2 (candidates, replays, comparison)
    ├── notebooks/            01 to 04: wrappers around the same commands
    └── RESULTS.md, RESULTS_V2.md, PLAN.md, docs/
```

The training package `svo` is shared: the API image copies exactly seven runtime modules (`__init__, model, monitor, preprocessing, artifacts, data, v2`) and refuses to start if their hashes differ from the hashes recorded in the bundle manifest. Scoring and monitoring logic therefore has a single implementation used by training, evaluation and the service.

3. System architecture

3.1 Deployment view

All services run on one host under Docker Compose and publish only on loopback. The browser talks to a single origin: in the container setup nginx serves the static console and proxies `/api` and `/health` to the API; in development the Vite dev server does the same proxying. The API therefore keeps CORS disabled by default.

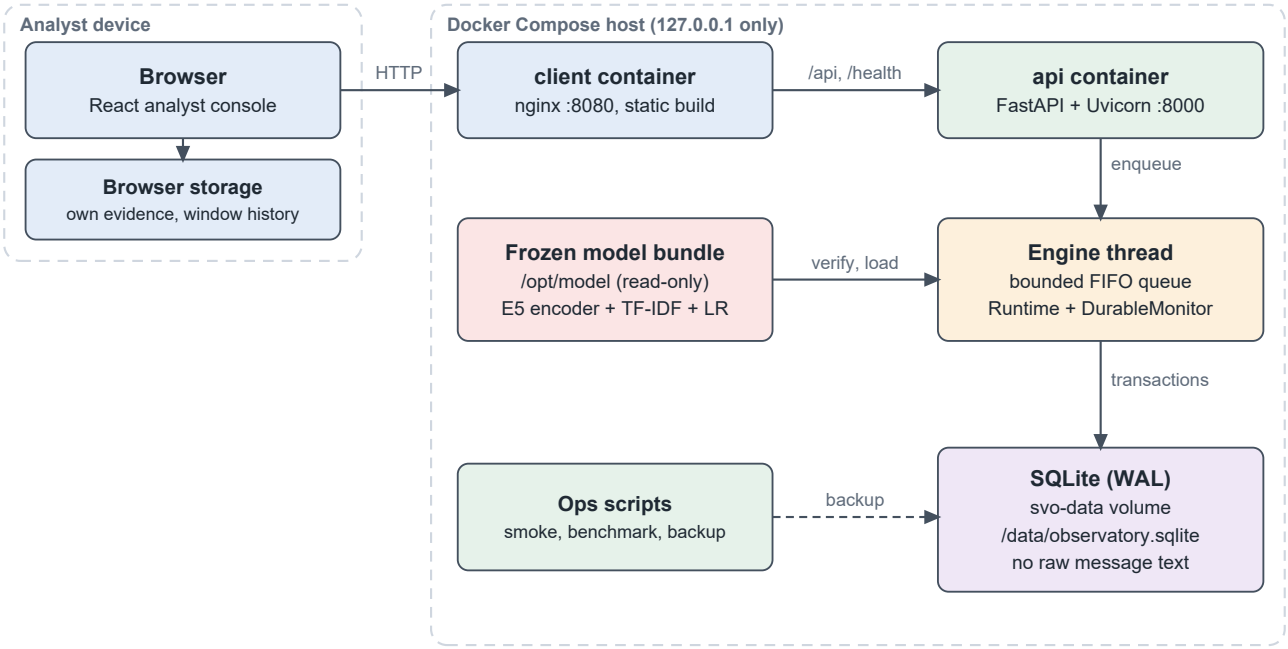


Figure 1. Deployment and component view.

3.2 Architectural principles

- **Single writer.** One Uvicorn worker, one bounded FIFO queue and one engine thread. Only that thread touches the model runtime and the SQLite connection, which makes monitoring order deterministic and removes locking from the scoring path. A file lock (`observatory.sqlite.lock`) stops a second process from opening the same database.
- **Frozen, verified model.** The bundle is immutable at runtime. Before any `joblib` load, the engine verifies the manifest version hash, every file checksum, the expected version, the representation allowlist and the source hashes of the runtime modules.
- **No raw text at rest.** The server stores SHA-256 hashes of normalised text and the prediction JSON, never the message. Logs record only request ID, route template, status and duration.
- **Idempotency everywhere.** Clients supply `event_id` and `review_id`. Retries with the same ID return the stored result; conflicting reuse returns 409. Accepted mutations complete even if the client disconnects.
- **Offline inference.** The E5 encoder ships inside the bundle; `HF_HUB_OFFLINE=1` and `TRANSFORMERS_OFFLINE=1` are set in the image. The service makes no outbound calls at runtime.
- **Honest outputs.** Thresholds, calibration status and limitations come from the bundle via `GET /api/v1/model`, so the console never shows claims that the loaded model cannot support.

3.3 Process and thread model

Element	Responsibility
Uvicorn event loop	HTTP parsing, the `Boundary` middleware, validation, health routes. Never blocks on the model.
`Engine.queue`	`queue.Queue(maxsize=SVO_QUEUE_CAPACITY)`, default 32. A full queue returns 429 with `Retry-After: 1`.
`svo-engine` thread	Acquires the DB lock, verifies and loads the bundle, runs a warm-up prediction, creates `DurableMonitor`, then serves queued calls (`predict, monitoring, clusters, detail, submit_review, reviews`).
`asyncio.shield`	Wraps each queued future so a disconnecting caller does not cancel work that was already accepted.
Shutdown	Stops accepting, drains the queue for up to 25 s; Compose gives 35 s before SIGKILL.
Storage failure	Any `sqlite3.Error` or `OSError` rolls back, marks the engine not ready and returns 503 until restart. Liveness stays up; readiness fails.

4. System flow

4.1 Startup sequence

- **1. Settings.** `Settings.from_env()` reads `SVO_*` variables and validates them (mode, credentials of at least 32 ASCII characters, explicit CORS origins, queue capacity 1 to 1024).
- **2. App factory.** `create_app()` registers the `Boundary` middleware, optional CORS, error handlers and routers, and starts the engine in the FastAPI lifespan.
- **3. Engine thread.** Creates the data directory, takes an exclusive non-blocking lock (`fcntl` on Linux, `msvcrt` on Windows), runs `verify_runtime()`, loads `Runtime` once and scores *"Service startup check."* to warm the encoder.
- **4. Storage.** `DurableMonitor` creates the core tables, runs migration to schema version 1, sets `synchronous=FULL`, checks disk space against the 16 MiB reserve and probes write access.
- **5. Ready.** `/health/ready` returns the loaded model version. The Docker healthcheck polls it every 5 s with a 120 s start period (cold start imports PyTorch and loads the encoder in about 25 s).

4.2 Prediction request lifecycle

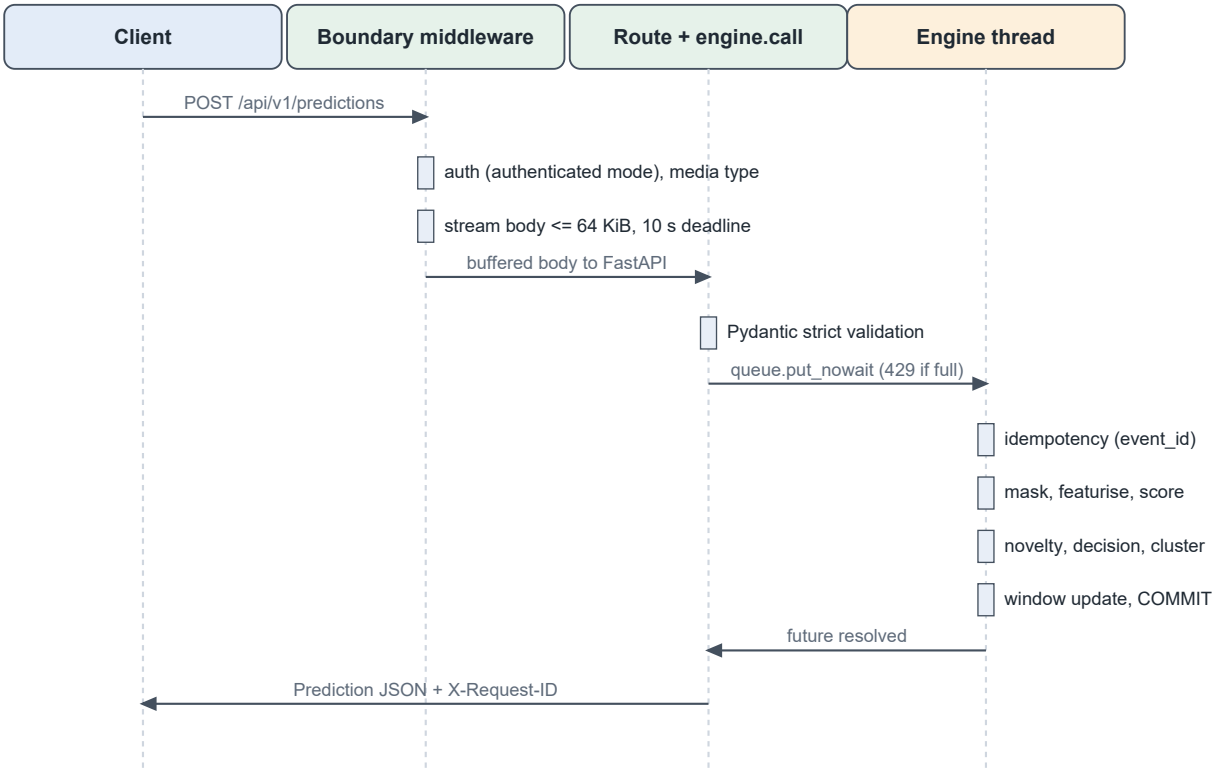


Figure 2. Sequence of a prediction request through the service.

Inside the engine thread, `DurableMonitor.predict()` runs these steps in one transaction:

- Check free disk space; compute `text_hash` (SHA-256 of NFC, whitespace-collapsed text).
- If the `event_id` already exists with the same hash and model version, return the stored prediction; otherwise 409.
- Score with `Runtime.predict_vectors()`: features, calibrated score, novelty, decision, nearest known scam family.
- If the message needs review, assign it to the most similar cluster (cosine ≥ 0.55) or open a new one, up to 100 clusters per version.

- Insert the event row. If the event count reaches a window boundary, compute the rolling window and the alert flag.
- Commit. On any failure, roll back so no partial cluster, event or window survives.

4.3 Scoring and decision logic

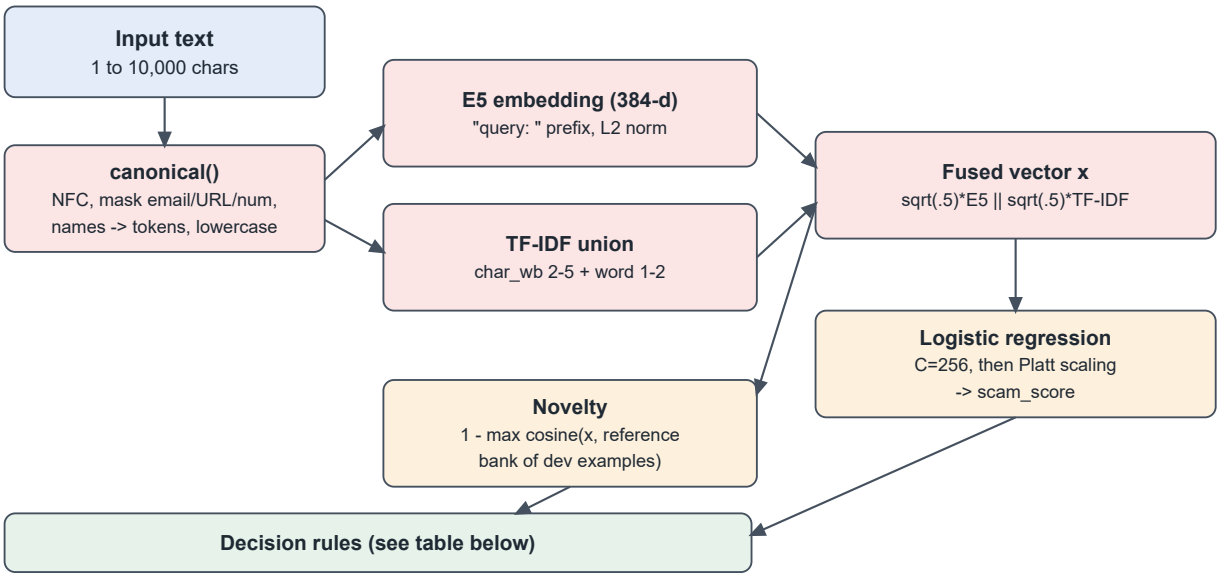


Figure 3. Feature extraction and scoring inside the v2 runtime.

The text is first passed through `svo.v2.canonical()`, the same masking applied to every training source: emails, URLs and domains become a link token, phone numbers and digits become a number token, capitalised words become an entity token, and the result is lowercased. Two representations are computed and fused with equal weight: a 384-dimensional multilingual-E5-small sentence embedding and a TF-IDF union of character (2 to 5, word-boundary) and word (1 to 2) n-grams. A logistic regression produces a raw score that Platt scaling turns into the calibrated `scam_score`.

Output field	Rule (thresholds from decision_config.json)
`novelty_score`	1 − max cosine similarity between the message vector and the reference bank (all development examples).
`is_unknown`	`novelty_score > 0.4806` (source-balanced 95th percentile of out-of-group novelty).
`decision = high_risk`	`scam_score ≥ 0.2893` (high-risk threshold).
`decision = low_risk`	`scam_score < 0.2893` (low-risk threshold) **and** not unknown.
`decision = review`	Everything else, including low scores on unfamiliar text.
`needs_review`	`is_unknown` or decision is not `low_risk`.
`nearest_known_family_or_null`	Family of the most similar scam example, only when the message is not unknown.
`cluster_id_or_null`	Assigned cluster when routed to review; null otherwise or when capacity is reached.

In v2 the high-risk and low-risk thresholds coincide (0.2893), so the review band exists only through novelty. Messages below the threshold that are unfamiliar are still sent for review.

4.4 Monitoring flow: novelty, clusters and drift

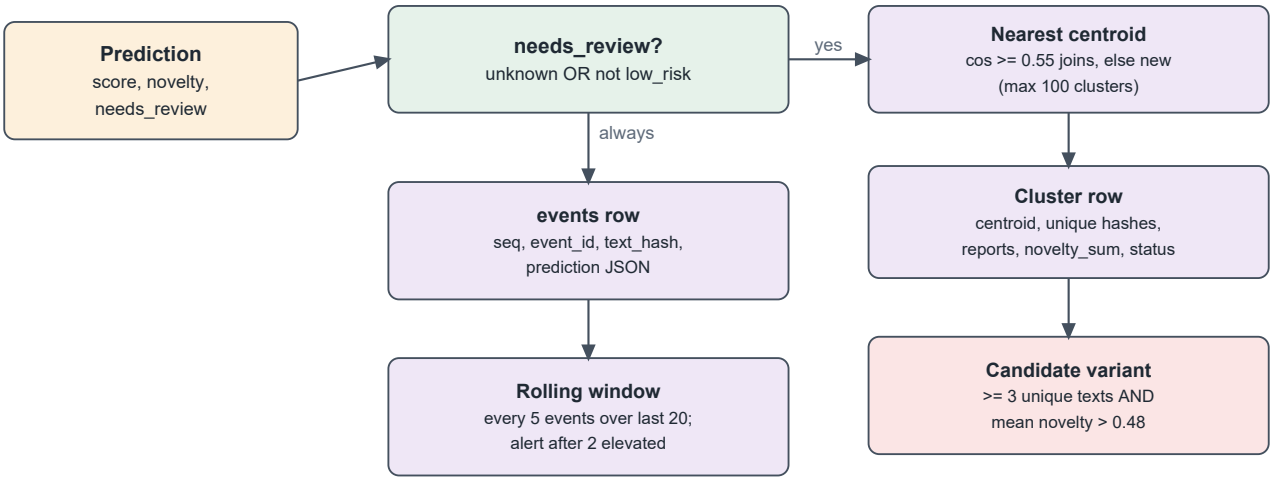


Figure 4. How each prediction updates clusters and rolling windows.

Clustering

- Only messages with `needs_review = true` are clustered.
- The message vector is compared with every cluster centroid of the same model version; the best match joins if cosine ≥ `cluster_similarity` (0.55, bounded below by the 99th percentile of unrelated development pairs).
- A new distinct text updates the centroid (running mean, re-normalised). Duplicate text increments `reports` without moving the centroid or increasing unique recurrence.
- At 100 clusters, new unmatched messages are still stored, with `cluster_capacity_reached = true`.
- A cluster is a **candidate variant** when it has ≥ 3 unique texts and its mean novelty exceeds the novelty threshold.

Rolling drift windows

- Window size 20 events, stride 5: a window is computed when the version's event count reaches 20, 25, 30, and so on.
- Each window stores the unknown fraction, review fraction and mean score of the last 20 events.
- A window is *elevated* when the unknown fraction exceeds `drift_unknown_threshold` (0.10).
- An **alert** fires when 2 consecutive windows are elevated. The threshold was chosen from 300 simulated normal streams so the normal alert rate stays at or below 5% (simulated rate 3.9%).

4.5 Analyst review flow

- The console lists clusters (`GET /api/v1/clusters`), filtered by status and paged by cursor.
- Cluster detail returns statistics and up to five representative event IDs. The browser shows the original text for any event it submitted itself, from its local evidence store; other events appear as IDs only.
- The analyst posts `{review_id, status, actor?}`. The engine fingerprints (cluster, status, actor, verified, version); a repeated `review_id` with the same fingerprint returns the stored result, a different one returns 409.
- The cluster status update, audit row (`reviews`) and idempotency row (`review_requests`) are written in one transaction. The response always carries `automatic_retraining: false`.
- In `local_demo` mode the actor is free text and marked unverified. In `authenticated` mode the actor is the credential alias and the body must not contain `actor`.

5. API reference

5.1 Endpoints

Method and path	Purpose	Engine call
GET /health/live	Process liveness (always public).	none
GET /health/ready	Readiness and loaded model version (public). 503 while starting or after storage failure.	none
GET /api/v1/model	Model version, representation, scope, calibration flag, mode, full decision config, limitations.	cached metadata
POST /api/v1/predictions	Score and record one message.	`predict`
GET /api/v1/monitoring	Event count, latest window, cluster/pending/candidate counts, free disk, schema version.	`monitoring`
GET /api/v1/clusters	Cluster queue. Query: `status`, `limit` (1 to 100, default 20), `after_id`.	`clusters`
GET /api/v1/clusters/{id}	Cluster details plus up to 5 representative event IDs.	`detail`
POST /api/v1/clusters/{id}/reviews	Submit an audited analyst decision.	`submit_review`
GET /api/v1/clusters/{id}/reviews	Audit history, cursor-paged.	`reviews`
GET /docs, /openapi.json	Offline documentation page and machine-readable contract.	none

5.2 Request and response examples

```
POST /api/v1/predictions
{"text": "The office opens tomorrow.", "event_id": "example-001"}

200 OK
{"event_id": "example-001", "scam_score": 0.04, "decision": "low_risk",
 "needs_review": false, "novelty_score": 0.31, "is_unknown": false,
 "nearest_known_family_or_null": "...", "cluster_id_or_null": null,
 "model_version": "svo-e2a46f504b19", "reference_version": "svo-e2a46f504b19",
 "scope": "unreviewed_mixed_source_demonstration", "cluster_capacity_reached": false}

POST /api/v1/clusters/7/reviews
{"review_id": "review-001", "status": "rejected", "actor": "local-analyst"}

200 OK
{"review_id": "review-001", "cluster_id": 7, "status": "rejected",
 "actor": "local-analyst", "actor_verified": false,
 "automatic_retraining": false, "model_version": "svo-e2a46f504b19"}
```

Numbers in the prediction example are illustrative. Field shapes match `schemas.py`.

5.3 Validation rules

- Strict Pydantic models: extra fields are rejected and no implicit type conversion happens.
- `text`: 1 to 10,000 characters, not blank, valid Unicode. Identifiers: 1 to 128 characters from `A-Z a-z 0-9 . _ ~ -`. `actor`: 1 to 80 characters from letters, digits, space, dot, underscore, hyphen.
- Bodies: uncompressed `application/json`, valid UTF-8, at most 64 KiB on the wire, read within 10 s. Non-standard JSON constants (NaN, Infinity) are rejected.

5.4 Error envelope and codes

```
{"error": {"code": "queue_full", "message": "Engine queue is full; retry with the same ID", "request_id": "3f2c..."}}
```

Status	Meaning	Typical code
400	Malformed JSON or invalid Content-Length	invalid_json, invalid_length
401	Missing or invalid credentials (authenticated mode)	unauthorized
404	Unknown route or cluster for this model version	http_error, cluster_not_found
408	Body not received within 10 s	body_timeout
409	Event or review ID reused with different content/model	event_conflict, review_conflict
413	Body larger than 64 KiB	body_too_large
415	Not uncompressed application/json	unsupported_media_type
422	Schema mismatch; actor sent in authenticated mode	invalid_request, actor_not_allowed
429	Engine queue full (Retry-After: 1)	queue_full
503	Engine not ready, storage unavailable or reserve exhausted	engine_unavailable, storage_unavailable, not_ready
500	Unexpected failure	internal_error

Every response carries an `X-Request-ID` header. Error messages and logs never echo message text.

6. Data storage

6.1 SQLite schema

The database lives at `/data/observatory.sqlite` in the named volume `svo-data`, in WAL mode with `synchronous=FULL` and a 2 s busy timeout. All monitoring state is keyed by model version, so a new model starts with a clean queue while event IDs stay globally unique.

Table	Columns	Purpose
events	seq (PK), event_id (UNIQUE), text_hash, version, created, prediction (JSON)	One row per scored report; idempotency key.
windows	seq (PK), version, unknown_fraction, review_fraction, score_mean, elevated, alert	Rolling drift windows.
clusters	id (PK), version, centroid (float32 BLOB), hashes (JSON, at most 1,000), reports, first_seq, last_seq, status, novelty_sum	Durable cluster state and review status.
reviews	id (PK), cluster_id, status, actor, created, version	Append-only audit trail.
review_requests	review_id (PK), fingerprint, audit_id, response (JSON)	Review idempotency (added by migration v1).

Indexes: `events(version, seq)`, `clusters(version, id)`, `reviews(version, cluster_id, id)`. `PRAGMA user_version` holds the schema version; a newer unsupported version stops startup.

6.2 Data that stays on the analyst device

Key	Storage	Content
svo.connection	sessionStorage	API base URL, API key (authenticated mode), analyst alias. Cleared when the tab closes.
svo.windows	sessionStorage	Up to 120 monitoring windows seen by this tab, per model version, for the trend chart.
svo.evidence	localStorage	Up to 500 submitted texts with their predictions, keyed by event ID. Can be disabled or cleared.

7. Security and privacy

7.1 Request boundary

`svo_api.auth.Boundary` is a raw ASGI middleware placed before FastAPI routing. It authenticates before reading the body, so unauthenticated clients cannot make the server buffer large payloads.

- Assigns a random request ID and adds it as `X-Request-ID` on every response.
- Public paths: `/health/live`, `/health/ready` and CORS preflight requests.
- Authenticated mode accepts `Authorization: Bearer` or `X-API-Key`, compared with `hmac.compare_digest` against every configured key; the matching alias becomes the actor.
- Checks media type and encoding, rejects oversize Content-Length early, streams the body with a hard 64 KiB cap and a 10 s timeout, and pre-parses JSON to reject invalid UTF-8 and non-standard constants.
- Logs one JSON line per request with route template, status and duration only.

7.2 Modes

Mode	Behaviour	Use
local_demo (default)	No key required; actor is free text, marked unverified. Published only on 127.0.0.1.	Local demonstration. Never expose through a tunnel, proxy or LAN port.
authenticated	All API and documentation routes require a key; health stays public. OpenAPI declares an API key scheme.	Any non-local use, behind a TLS-terminating proxy. Keys: unique, ASCII, at least 32 characters, injected via a secret store.

7.3 Container hardening

- Base image pinned by digest; dependencies pinned by lock file.
- Runs as UID/GID 10001; model, runtime source and API code made read-only during build.
- Compose: read-only root filesystem, 32 MB `/tmp` tmpfs, `cap_drop: ALL`, `no-new-privileges`, `init`, 2 CPUs, 2 GiB memory, loopback-only ports.
- Build context is an allowlist (`.dockerignore`): no datasets, caches, virtual environments or evaluation databases.
- nginx (client): unprivileged image, CSP `default-src 'self'`, `X-Frame-Options DENY`, `nosniff`, `no-referrer`, 64 KiB body cap, immutable caching for hashed assets only.

7.4 Artifact integrity

The manifest version is `svo-` plus the first 12 hex characters of the SHA-256 of the manifest content. `verify_bundle_portable()` recomputes it, checks every listed file hash, rejects paths outside the bundle, and accepts Windows-sealed manifests with backslash paths. Checksums detect corruption and tampering but do not prove who produced the joblib file, so only team-controlled bundles should be loaded.

8. Analyst console (frontend)

The console is a single-page React application that covers the full detect → monitor → respond loop. It reads every threshold and wording choice from `GET /api/v1/model`, so a model change never leaves stale claims on screen.

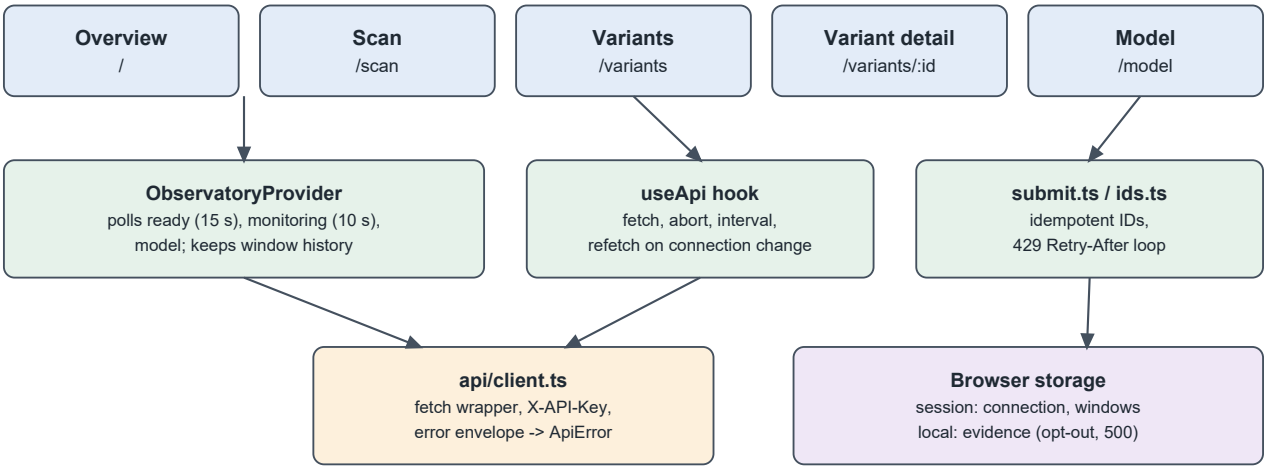


Figure 5. Frontend structure: pages, shared state and API access.

8.1 Pages

Page	Purpose	API
Overview (/)	Drift alert banner, unfamiliar-message rate per window (session chart), queue counts, pending candidate variants.	monitoring, clusters?status=pending
Scan (/scan)	Score one message, or replay a batch line by line in order so that monitoring windows behave as in production.	POST predictions
Variants (/variants)	Cluster queue filtered by status, candidates-only toggle, sorting, cursor pagination.	clusters
Variant detail	Progress toward the candidate rule, representative reports (local text where available), review form, audit trail.	clusters/{id}, reviews
Model (/model)	Frozen thresholds, monitoring settings, deployment mode, limitations.	model

8.2 Design decisions

- **Scores against thresholds.** Score and novelty are drawn on separate scales marked with their thresholds; they mean different things.
- **Low risk is not green.** Decisions use rose, amber and neutral slate, always with an icon and label.
- **Idempotent retries.** Each scan and review gets a client ID (`web-`, `batch-`, `review-` prefix plus time and 8 random bytes). Retries after network errors, timeouts, 429 or 5xx reuse it; 429 is retried automatically after `Retry-After` (up to 4 attempts). Editing the text creates a new ID.
- **Evidence stays on the device.** The server never stores text, so the browser keeps its own copy keyed by event ID.
- **No secrets in the bundle.** API keys are entered at runtime and kept in sessionStorage; requests send `X-API-Key`.
- **Accessibility.** Semantic landmarks, `role="meter"` scales, `aria-live` results, visible focus, reduced-motion support, checked at 360 px width.

9. Machine learning pipeline

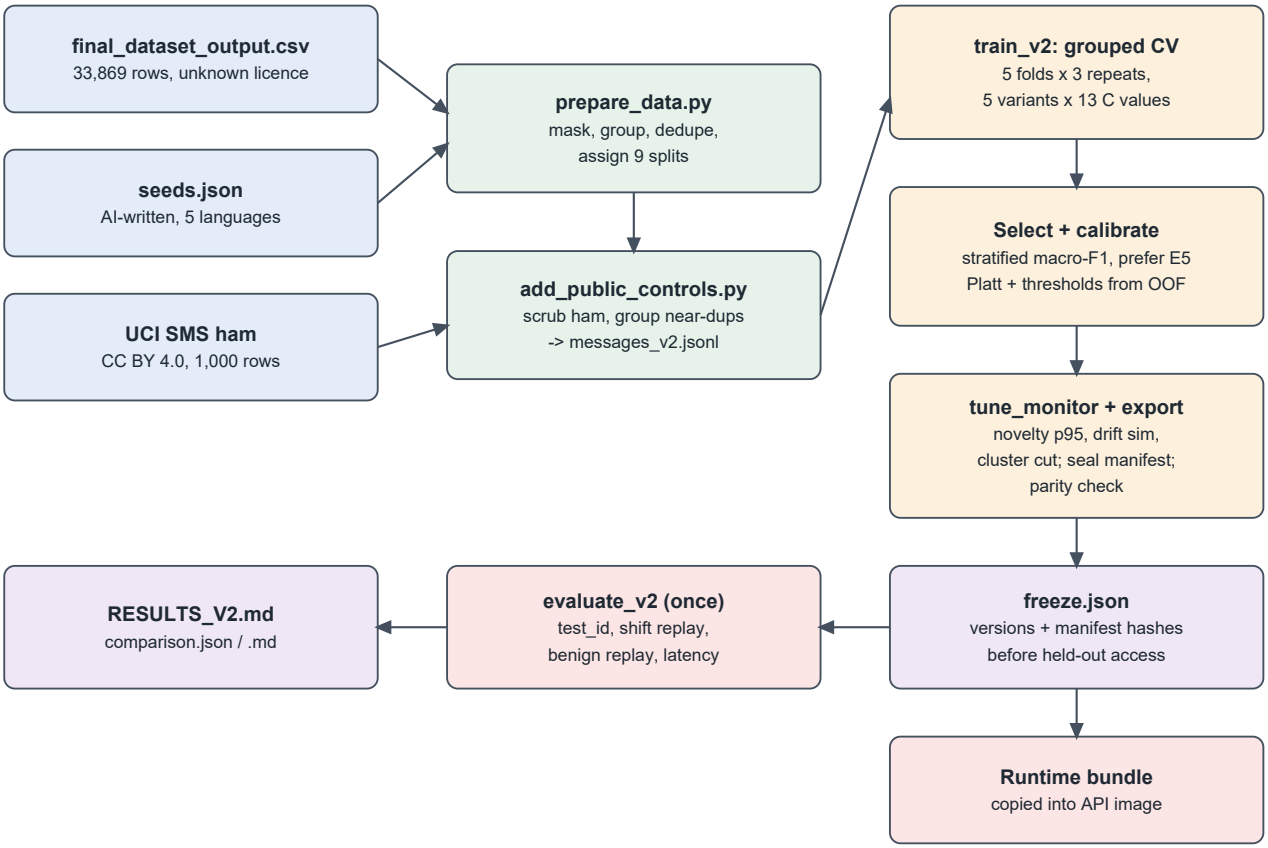


Figure 6. v2 training, selection, freeze and evaluation flow.

9.1 Data sources

Source	Content	Status
final_dataset_output.csv	33,869 rows, mostly English, scam categories only, no legitimate class. At most 400 masked English rows from banking, delivery, government and telecom are admitted.	User-supplied; provenance and licence unresolved.
seeds.json	AI-written fictional scam and legitimate scenarios in English, Sinhala, Tamil, Singlish and Tanglish, each with a "Notice:" prefix copy.	Unreviewed synthetic data.
UCI SMS Spam Collection (ham only)	1,000 real legitimate SMS, masked like the CSV and grouped into 993 near-duplicate groups. UCI spam is not used because spam is not fraud.	CC BY 4.0.

9.2 Data preparation and splits

- `prepare_data.py` masks CSV rows, builds synthetic rows, consolidates related templates by fingerprint and cosine similarity, and assigns whole scenario groups to splits. It refuses to overwrite its output.
- `add_public_controls.py` adds UCI ham into a new file, `messages_v2.jsonl`.
- `svo.data.load_data()` enforces required fields, unique IDs, no group crossing splits, no exact duplicate text, both classes in key splits, and that the held-out mechanism family never appears in training. Unreviewed data requires `--simulation`.

Split	Scam	Legit	Groups	Role
train	318	662	800	Development pool (pooled for grouped CV in v2)
val_select	59	167	186	Development pool
val_cal	68	127	140	Development pool
shift_dev	20	20	2	Selects cluster similarity
test_id	85	174	196	Held-out test (evaluated once)
shift_demo	60	60	6	Held-out shifted replay (unseen remote-access family)
update_train / update_val	10 / 10	10 / 10	1 / 1	Reserved for controlled update (unused)
post_update_test	20	20	2	Held-out, for a future update

Language counts: en 1,500; si 100; ta 100; singlish 100; tanglish 100 (from the v2 split manifest).

9.3 Training (svo.train_v2)

- **Shared masking.** `canonical()` is applied to every source at training and inference, removing the v1 shortcut.
- **Candidates.** `tfidf_char_raw` (ablation, excluded from selection), `tfidf_char`, `tfidf_union`, `e5`, `e5_tfidf`; C from 0.25 to 1024 (13 values).
- **Grouped CV.** `StratifiedGroupKFold`, 5 folds × 3 repeats over train + val_select + val_cal; vectorizers re-fitted per fold. E5 embeddings are cached by text, encoder revision and settings.
- **Weighting.** Prefix copies share one unit of weight; each (synthetic?, label) cell gets equal total weight.
- **Selection.** Highest mean of per-source macro-F1 (synthetic vs real) on out-of-fold scores; ties by PR-AUC, then smaller C. The best E5 variant is preferred if within 0.02 of the top, because its encoder also serves semantic monitoring.
- **Calibration and thresholds.** Platt scaling on out-of-fold scores. High-risk threshold: highest recall with precision ≥ 0.90. Low-risk threshold: largest cut leaking at most 2% of development scams, capped at the high-risk threshold.
- **Monitor tuning.** Novelty threshold from out-of-group nearest neighbours (source-balanced 95th percentile); drift threshold from 300 simulated 120-message normal streams; cluster cut from shift_dev pair F1, never below the unrelated-pair 99th percentile.
- **Export.** Writes `model.joblib`, `decision_config.json`, `model_card.md` and the encoder, seals the manifest (data hash, source hashes, packages, encoder revision), and checks that a reloaded bundle reproduces scores within 1e-5.
- **Freeze.** `freeze.json` records every candidate version and manifest hash before any held-out data is read.

9.4 Evaluation (svo.evaluate_v2, once only)

All frozen candidates and the shipped v1 bundle are scored on identical held-out rows. The run produces test metrics with group bootstrap confidence intervals, language and source slices, a shifted replay and a benign-only replay through the real monitor, and latency. A `FINAL_EVALUATION_STARTED.json` marker prevents accidental repeat evaluation.

9.5 Results

Model	Macro-F1 [95% CI]	PR-AUC	Precision / recall	Legit FPR	Review /100	p95 ms	Size
v1 shipped	0.382 [0.28-0.46]	0.655	33.9% / 69.4%	66.1%	100	3.8	0.5 MB
tfidf_char_raw	0.904 [0.84-0.98]	0.969	87.2% / 96.5%	6.9%	38	13	1.8 MB
tfidf_char	0.915 [0.86-0.98]	0.951	84.4% / 95.3%	8.6%	41	12	1.7 MB
tfidf_union	0.915 [0.86-0.97]	0.932	81.2% / 96.5%	10.9%	41	17	2.3 MB
e5	0.913 [0.86-0.98]	0.953	77.3% / 100%	14.4%	52	41	473 MB
e5_tfidf (served)	**0.932 [0.88-0.99]**	0.962	83.2% / 98.8%	9.8%	40	52	477 MB

Test set: 259 messages, 201 groups. Language macro-F1 for the served model at 0.5: en 0.955, si 0.829, singlish 0.829, ta 1.000, tanglish 0.625; each non-English slice has only 3 scenarios. Confidence intervals overlap for all v2 models, so their ranking is not statistically established. No v2 model detected the held-out shift.

9.6 Why v2 replaced v1

v1 problem	v2 fix
Only scam CSV rows were masked, so mask tokens became the top scam features (78% of real legitimate SMS flagged).	One `canonical()` masking for all sources, at training and inference.
No real legitimate data; legitimate metrics rested on 2 to 3 synthetic groups.	1,000 real UCI ham messages added.
C and thresholds fitted on tiny partitions; uncalibrated scores with a fixed 0.2 cut; 100% review workload.	Pooled grouped CV, Platt calibration, data-driven thresholds; workload 40 per 100.
Monitor thresholds fitted in-sample; 33% false alerts on benign-only replay; cluster cut at grid edge (0.3).	Out-of-group novelty, simulated drift threshold, cluster cut ≥ unrelated p99 (0.55).
Windows portability: CRLF rewrote bundle files, `resource` import crashed.	UTF-8/LF IO, `.gitattributes`, portable verification.

10. Deployment and operations

10.1 Running the system

```
# API only
docker compose up --build -d --wait          # http://127.0.0.1:8000/docs

# API + analyst console
docker compose --profile frontend up --build -d --wait   # http://127.0.0.1:8080

# Smoke test and a manual prediction
python3 app/server/scripts/smoke.py
curl -s http://127.0.0.1:8000/api/v1/predictions \
  -H 'Content-Type: application/json' \
  -d '{"text":"The office opens tomorrow.","event_id":"example-001"}'

# Frontend development (Node 20+), proxies /api and /health to 127.0.0.1:8000
cd app/client && npm install && npm run dev  # http://127.0.0.1:5173
```

Build prerequisite. The E5 encoder inside the v2 bundle (about 450 MB) is git-ignored because it exceeds GitHub's 100 MB file limit, so a fresh clone cannot build the API image. Transfer the complete bundle privately and verify it with `app/server/scripts/verify_runtime.py`. Re-downloading the encoder does not reproduce the manifest hashes. The smaller lexical fallback `tfidf_char` (1.7 MB, `svo-b3ab151e73a2`) can be served by changing `SVO_MODEL_DIR` and `SVO_EXPECTED_MODEL_VERSION` and adding that bundle to the build context.

10.2 Configuration

Variable	Default	Meaning
SVO_MODE	local_demo	local_demo or authenticated
SVO_CREDENTIALS_JSON	{}	JSON map of analyst alias to secret key; required in authenticated mode
SVO_ALLOWED_ORIGINS	empty	Comma-separated exact origins for CORS; disabled when empty
SVO_QUEUE_CAPACITY	32	Waiting engine operations before 429
SVO_MODEL_DIR	/opt/model (container)	Bundle directory
SVO_EXPECTED_MODEL_VERSION	svo-e2a46f504b19	Startup fails if the bundle version differs
SVO_DB_PATH	/data/observatory.sqlite	SQLite database path
SVO_API_PROXY_TARGET (client dev)	http://127.0.0.1:8000	Vite proxy target
VITE_SVO_API_BASE_URL (client build)	empty (same origin)	Default API base URL in the console

10.3 Backup, restore and rollback

```
docker compose exec api python scripts/backup.py /data/observatory.sqlite /data/backup.sqlite
docker compose cp api:/data/backup.sqlite ./observatory-backup.sqlite
```

- The backup uses SQLite's online backup and refuses to overwrite an existing file.
- Restore: stop the API, mount the volume in a maintenance container, keep the current DB/WAL/SHM as a recovery copy, replace the database, remove stale WAL/SHM, restore UID/GID 10001 ownership, then start.

- Rollback: keep the previous image tag and a compatible database backup. No hot model replacement; model versions isolate monitoring state.
- `docker compose down` keeps data; `down -v` deletes all service state.
- Watch `disk_free_bytes`: below 16 MiB, readiness fails and engine work is rejected until repair and restart. No automatic record expiry exists, because deleting records would change idempotency and monitoring history.

10.4 Scaling constraints

Do not run multiple workers or replicas against the same volume; the file lock makes a second engine refuse to start. Throughput is bounded by the single engine thread. On the Windows development host v2 measured 281 / 336 ms (p50 / p95) HTTP latency at concurrency 4 under memory pressure; v1 measured 29 / 33 ms in the Linux container. v2 container latency still needs to be measured.

11. Testing and verification

Suite	What it checks
app/server/tests/test_contract.py	Endpoints, validation, error envelope, authentication, CORS, body limits, queue saturation.
app/server/tests/test_parity.py	API output equals the frozen runtime on multilingual fixtures and the private shift replay, in order.
app/server/tests/test_persistence.py	Idempotency, conflicts, rollback on injected faults, restart persistence, backup/restore, version isolation, single-writer lock.
app/server/tests/test_container_smoke.py	Real HTTP smoke against a running container (needs SVO_TEST_URL).
app/server/scripts/verify_runtime.py	Bundle manifest, file hashes and runtime source hashes (also run during docker build).
training/tests/test_pipeline.py, test_v2.py	Data admission, leakage checks, masking, weighting, export parity.

```
python3 -m venv app/server/.venv
app/server/.venv/bin/python -m pip install -r app/server/requirements-dev.lock
PYTHONPATH=app/server/src:training/src app/server/.venv/bin/python -m pytest app/server/tests -q
PYTHONPATH=training/src training/.venv/bin/python -m unittest discover -s training/tests
```

11.1 Verification status (v2, 17 October 2026)

Check	Result
API test suite against v2 bundle	15 passed, 1 skipped (container smoke)
Prediction and monitoring parity	Passed
Windows-sealed manifest, path traversal and tamper rejection	Passed
Representation allowlist and v2.py source hash	Passed
Local HTTP smoke	Passed
Resident memory after 200 predictions	About 1.19 GB (peak 1.23 GB)
Docker build, compose --wait, read-only/non-root and offline container for v2	**Not yet re-verified** (Docker absent on the verification host; v1 passed all of these)

12. Limitations and next steps

12.1 Known limitations

- **Precision target missed.** High-risk precision is 83.2% against a 90% target; 15 of 17 false positives are synthetic formal notices.
- **Drift not detected.** The held-out remote-access scams were all caught as high risk, but none was "novel", so no alert or candidate fired. The novelty threshold is dominated by naturally unique UCI chat.
- **Data.** CSV provenance and licence are unresolved; synthetic data is unreviewed; real legitimate data is informal chat; each local-language slice has very few scenario groups.
- **Calibration.** Scores are calibrated under a balanced class prior, not deployment prevalence.
- **Masking.** Collapsing capitalised words into one entity token removes meaning; very short texts can score identically.
- **Operations.** Single engine thread; no automatic retention; window trend in the console is per-tab only; v2 container not yet re-verified.
- **Repository.** Contains training data and learned artifacts; keep it private.

12.2 Recommended next steps

- 1. Collect or write 50+ formal legitimate scenarios (OTP warnings, delivery updates, bank notices) in all five languages to fix precision.
- 2. Re-prepare from the raw CSV with light masking (real PII only) applied to all sources; use far more than 400 scam rows.
- 3. Apply per-source novelty thresholds and leave-one-family-out simulation, then build a new locked shift replay.
- 4. Consider encoder fine-tuning only after steps 1 to 3.
- 5. Re-verify the v2 image on a Docker host and measure container latency; update the root README to v2.

12.3 Glossary

Term	Meaning
Bundle	Frozen, hash-sealed directory with model, decision config, model card and (for E5) encoder.
Canonical text	Output of the shared masking function applied before any feature extraction.
Novelty	Distance to the nearest development example in feature space. A review signal, not a fraud probability.
Unknown / unfamiliar	Novelty above the novelty threshold.
Window	Statistics over the last 20 events, computed every 5 events.
Drift alert	Two consecutive windows with an unfamiliar fraction above the drift threshold.
Cluster	Group of review-routed messages with similar vectors.
Candidate variant	Cluster with at least 3 distinct texts and mean novelty above the threshold.
Scenario group	All translations and variants of one scenario; never split across partitions.
OOF	Out-of-fold scores from grouped cross-validation, used for selection and calibration.